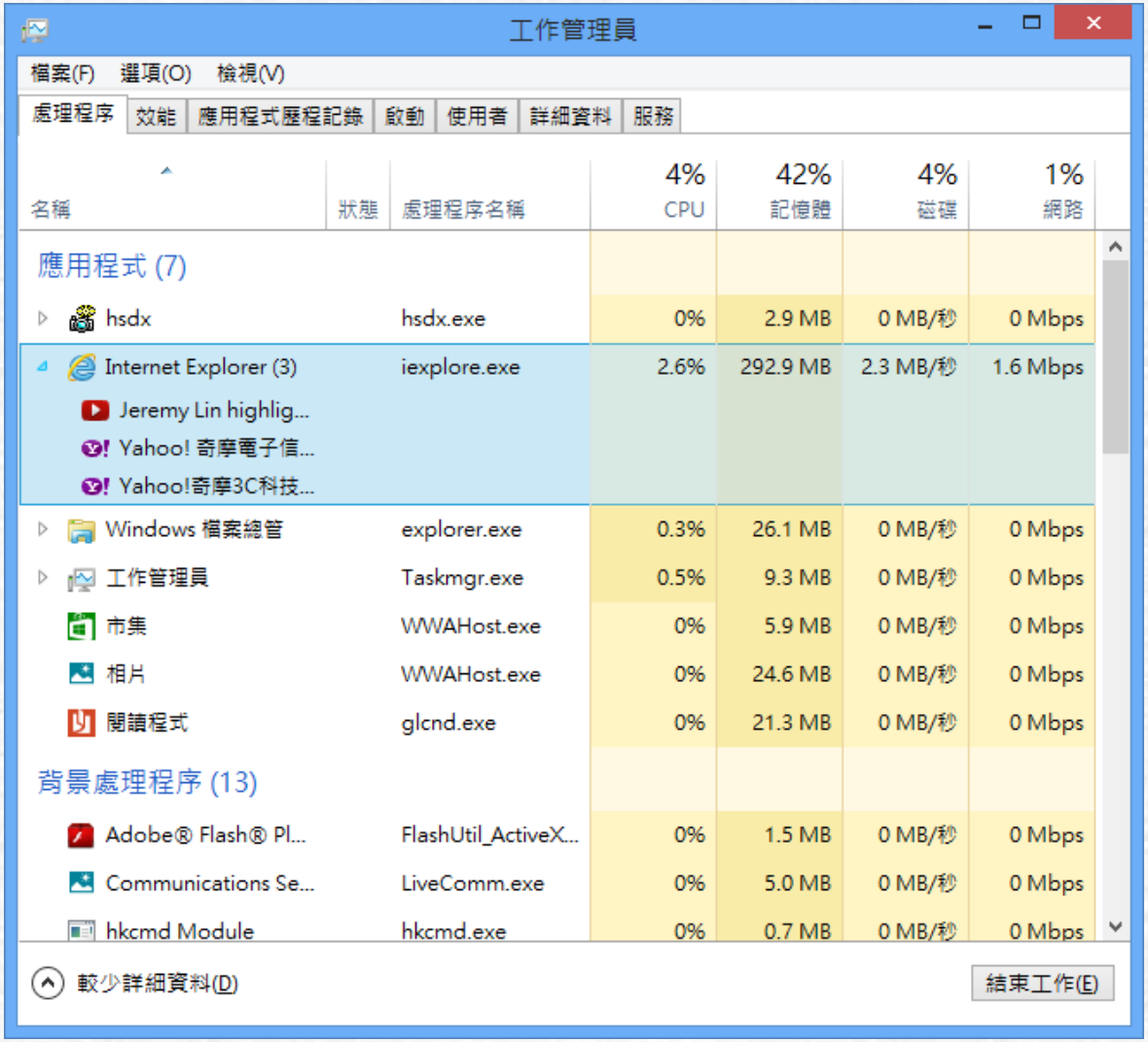
Chapter 2 Process Management

2-1 What is a Process (trip)?

- Instructions in a computer system are stored in auxiliary memory as static program files
- When a program is loaded into the system for execution, it is called a process
- The trip includes: :
  - Program section
  - Data section
  - Stack sections
  - Control information

Figure 2-1 IE program example



Why do you need a Process ?

- Process = program?

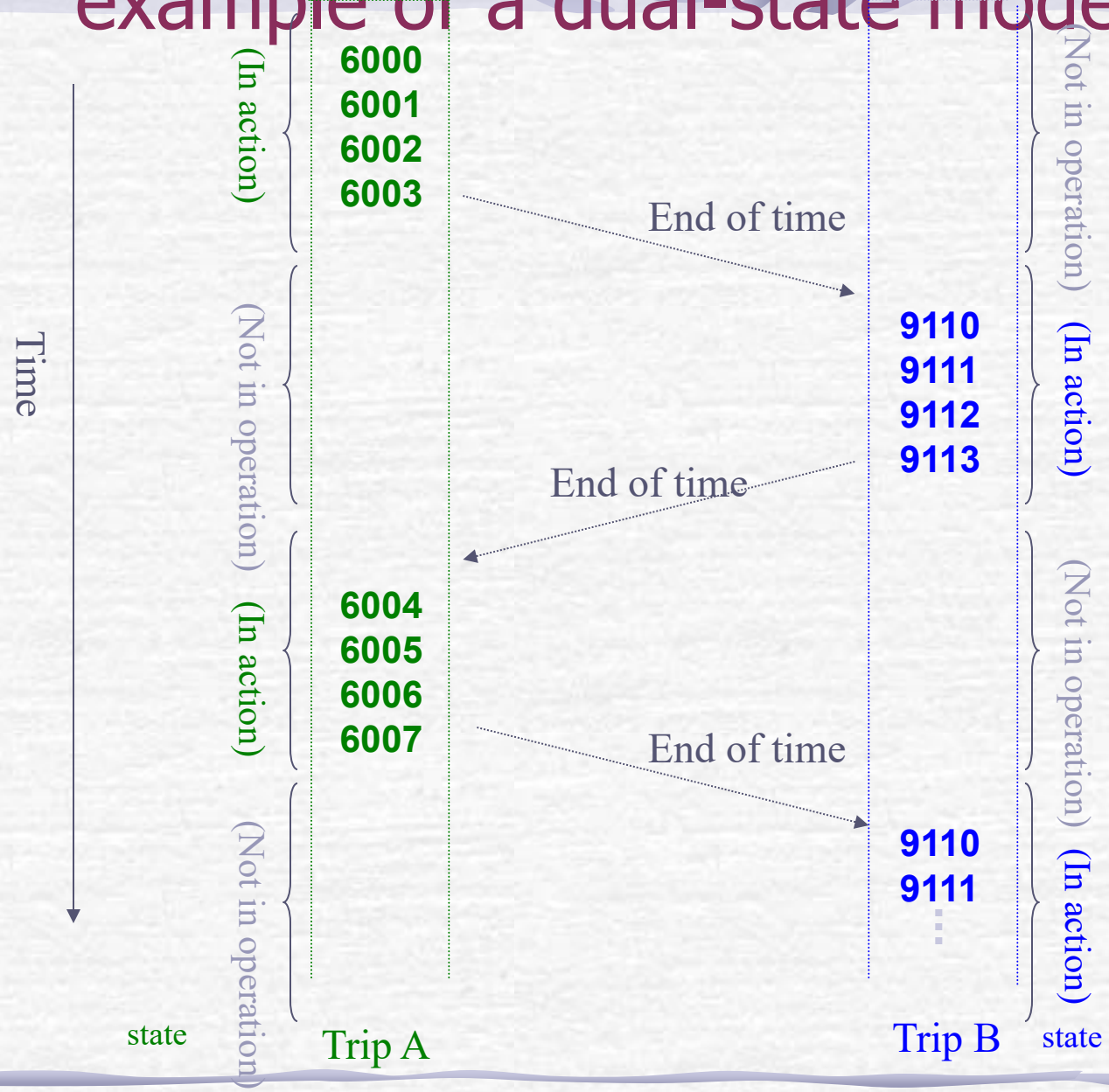
The main difference between a process and a program

- The process is the active entity, and the program is the passive entity
- Processes are temporary, and programs are persistent
- The process is different from the content of the program

2-2 Process status

- During the process execution, it may go through several different states
- The status of the process reflects the current actions of the trip and what can be done
  - What can be done in each state is different
  - Transitions between different states occur due to certain conditions

The simplest process state model - an example of a dual-state model



State transitions for dual-state models

- In practice, it is up to the dispatcher to decide which process can then enter the running state
- In a two-state model, when a new process is created, it first enters the system queue in an inactive state, waiting for an opportunity to run
- When a currently running process is interrupted, the dispatcher chooses one of the processes waiting in the queue to execute

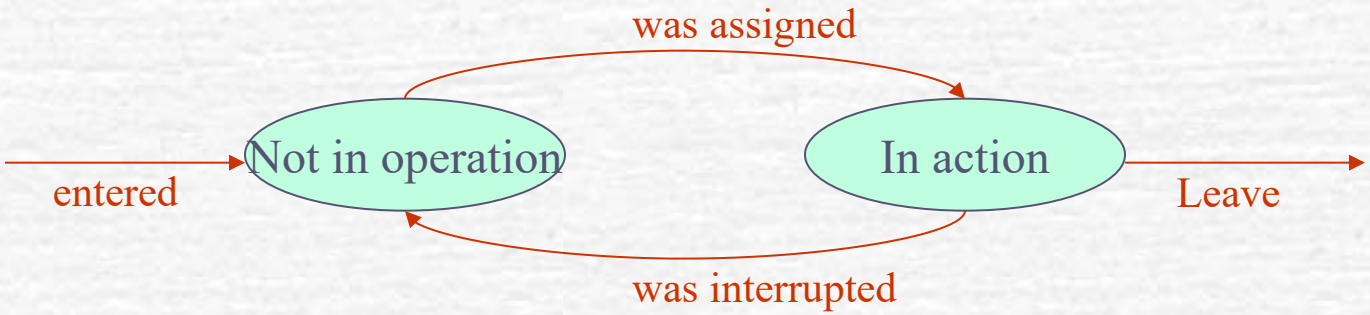
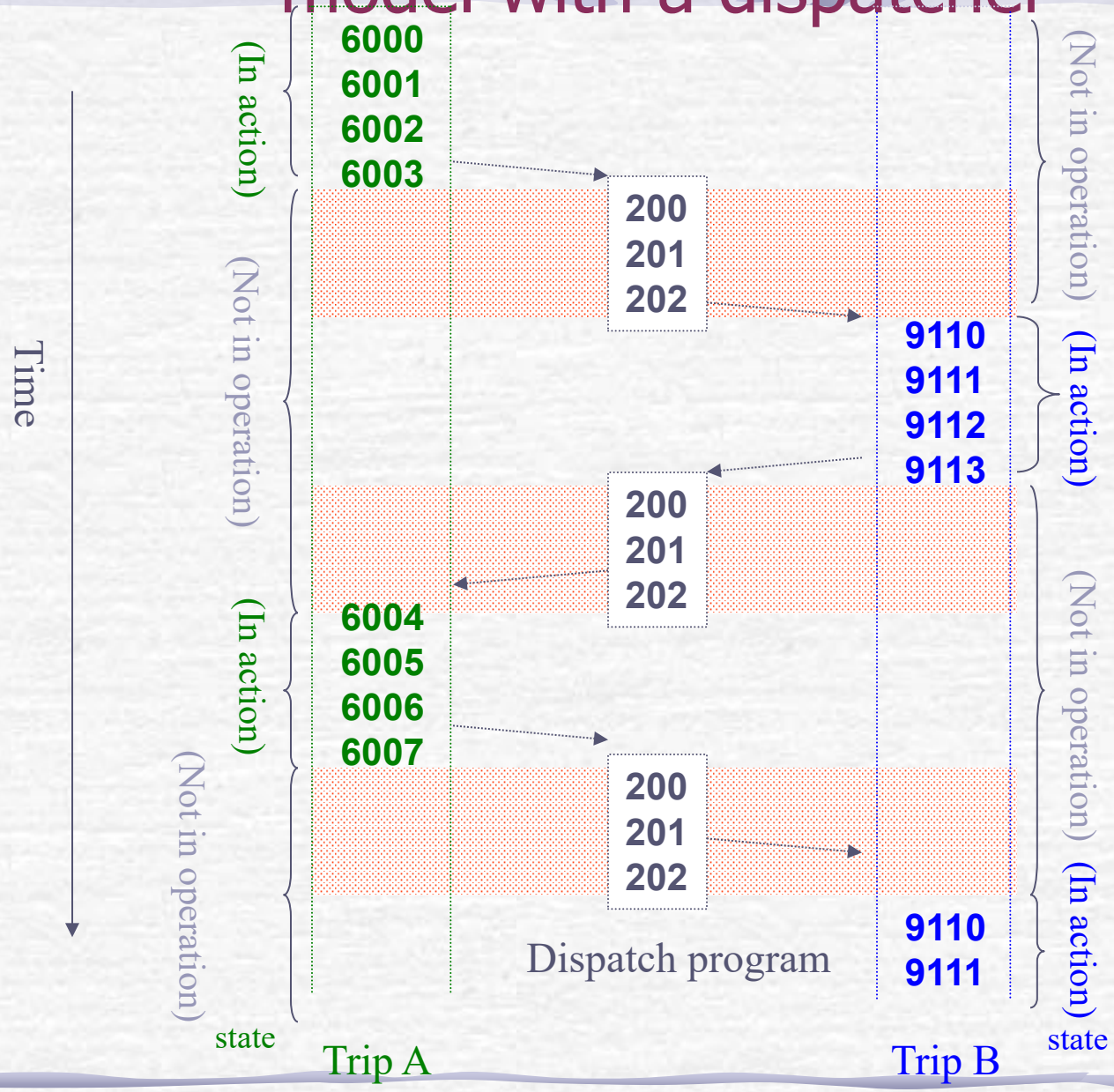


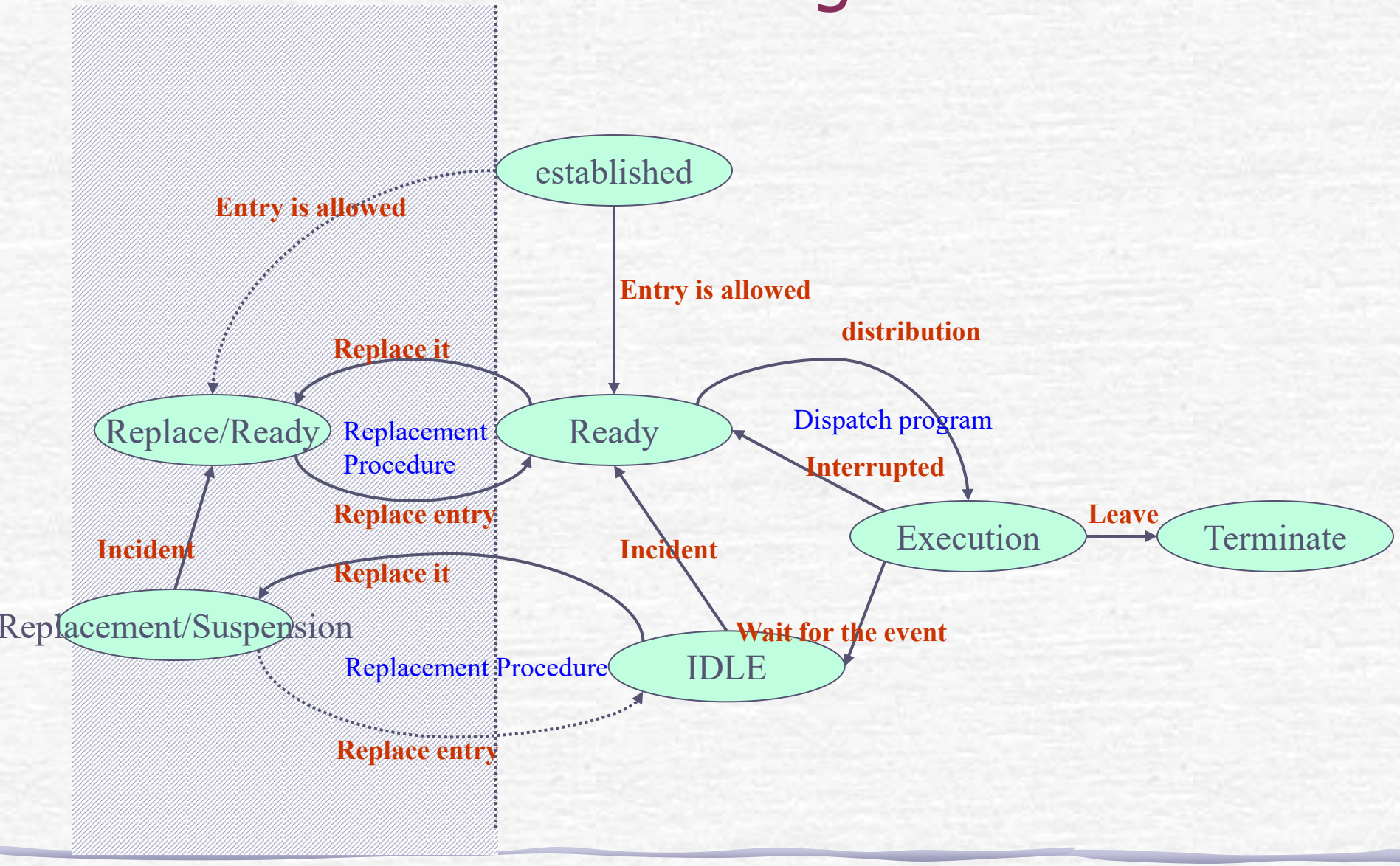
Figure 2-3 An example of a two-state model with a dispatcher



Long-term and short-term scheduling

- Scheduling: In a multitasking system, the process of deciding which process can gain CPU control
- Long-term scheduler：
  - It is used to determine which tasks (itineraries) can participate in the competition for system resources; Commonly found in traditional batch operating systems
- Short-term scheduler：Also known as dispatch procedures
  - It is responsible for allocating the CPU to processes that have entered memory and are ready to be executed
  - Interim scheduler: Also known as a replacement program

Figure 2-5 Complete process state conversion diagram



Classroom Exercises - Schedule State Conversion

範例補充包-Extra~

- Suppose there is a process performing an I/O operation. While waiting for the calculation to complete, the system was moved to auxiliary memory by the replacement program because the multi-engineering of the system was too high.
- When this process is still in the auxiliary memory, its I/O operation is completed.
- What states did it go through until it was re-executed?

Practice Answers

- Execution→IDLE→Replacement/IDLE→Replacement/Ready→Ready→Execution

2-3 Creation of the Process

- When a process is in the creation state, it means that the operating system has finished creating the process control information, but has not yet moved the process into memory
  - This design technique helps the operating system differentiate between the task of process control and memory allocation

The timing of the new Process :

- User login
- The user runs a program
- Request the system to provide services
- generated by an existing itinerary→Fathers Process

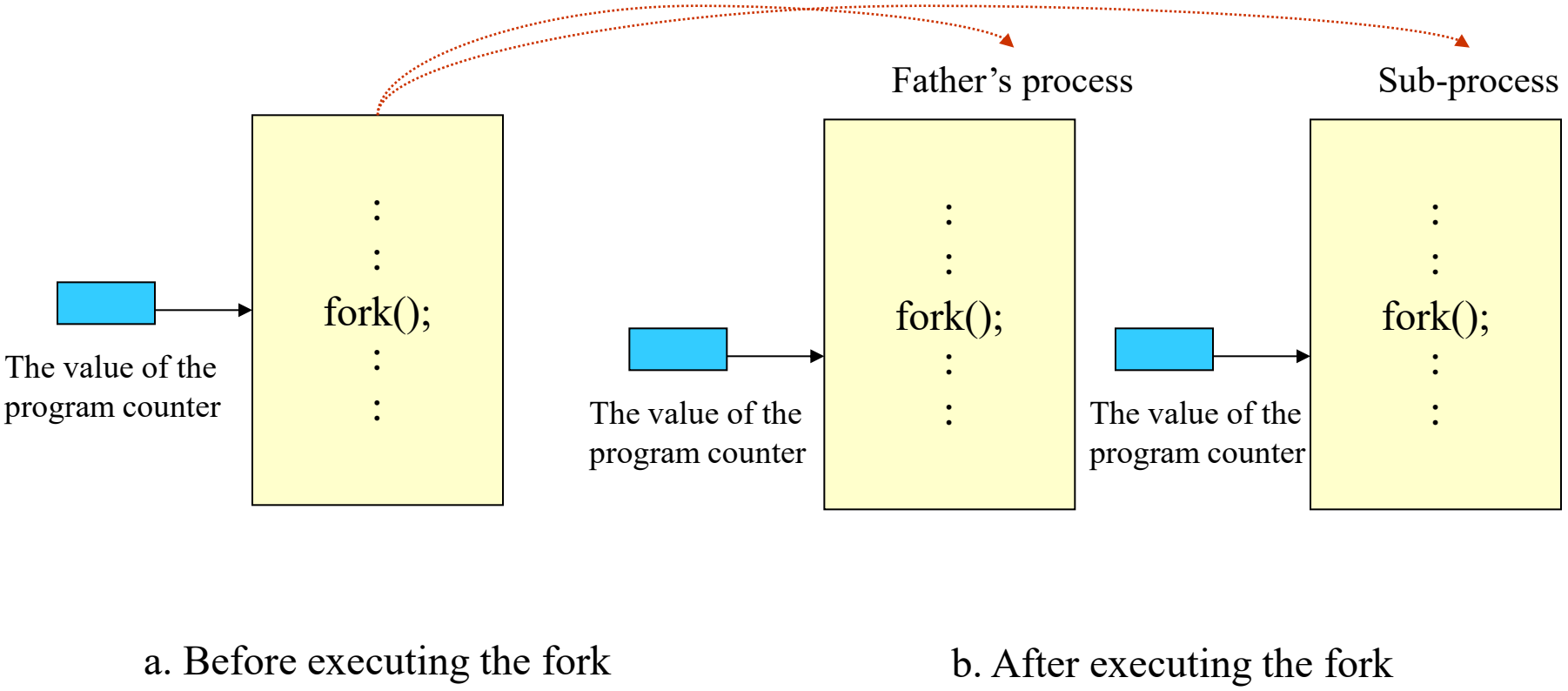
The end of the Process

- When a process is in a terminated state, it means that the process can no longer be executed, but some of its related information and tables can still be temporarily saved in the operating system
- Possible reasons for the end of the tour：
  - Mission completed
  - Protect errors
  - Calculation error
  - I/O failure
  - External force terminated
  - The parent Process is terminated
  - Parent Process requirements

Practical discussion - how Unix's parent routine spawns child routines

- In Unix, the only way for an existing process to generate a new one is to use a fork system call
- Fork will "vegetately propagate" the executing process, duplicate the address space of the parent process, and create an identical copy
- When the fork is executed, it returns to the parent and child processes, respectively
- The itinerary relies on the return value of the fork to determine whether it is a "deity" or a "clone"
- Once the clone is divided into two, one of the processes can run another brand new program through the exec series system call, transforming into a completely different process to complete other tasks

Figure 2-6 How fork works



a. Before executing the fork

b. After executing the fork

Fork program example

```
main()
{
        int child_pid;
        child_pid = fork();
        if (child_pid == -1) { /* Indicates fork failure*/
                perror ("can't fork");
                exit(1);
        }

        if (child_pid == 0) { /* indicates that it is a sub-itinerary*/
                ……
        } else { /* Indicates that it is the parent itinerary*/
                ……
        }
}
```

Use a fork to execute user commands

- The shell uses the fork to generate a new process, which is then used to execute the user's program
- Using the ps –al command to check the process status, you will find that the parent process of these commands or processes is a shell
- For example, after executing a.out under the shell, use ps -al：

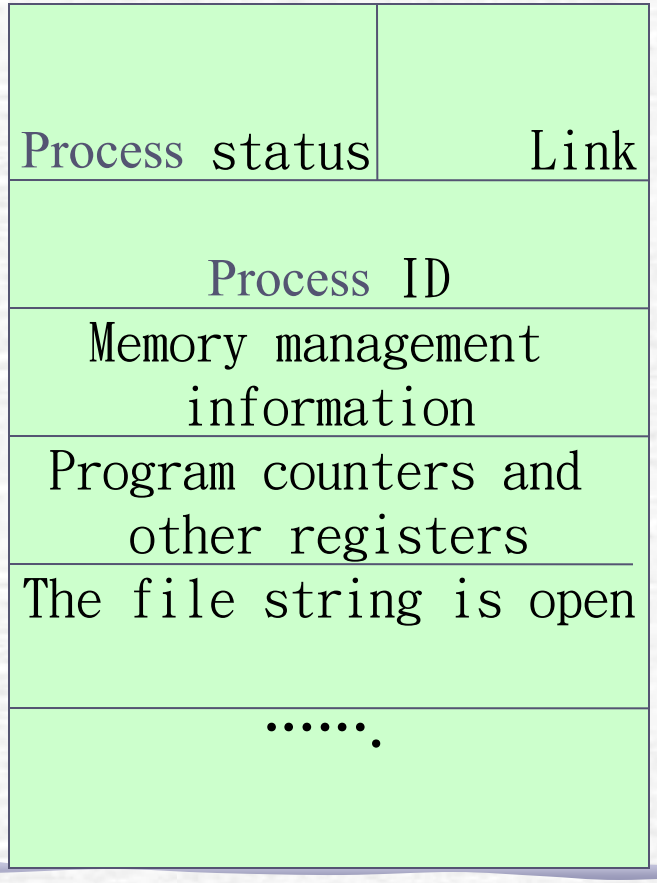
```
F   UID   PID  PPID %C PRI NI   SZ  RSS    WCHAN S TT       TIME COMMAND
8  1037 24198 24196  0  49 20 2760 2216 721d22e6 S pts/2    0:00 -tcsh
8  1037 24234 24198  0  59 20  960  496   709b6d26 S pts/2    0:00 ./a.out
8     0 24235 24198  0  49 20 1008  840            O pts/2    0:00 ps –al
```

Shell, ID is 24198

subroutine of shell,
The parent trip ID is 24198

2-4 Stroke control

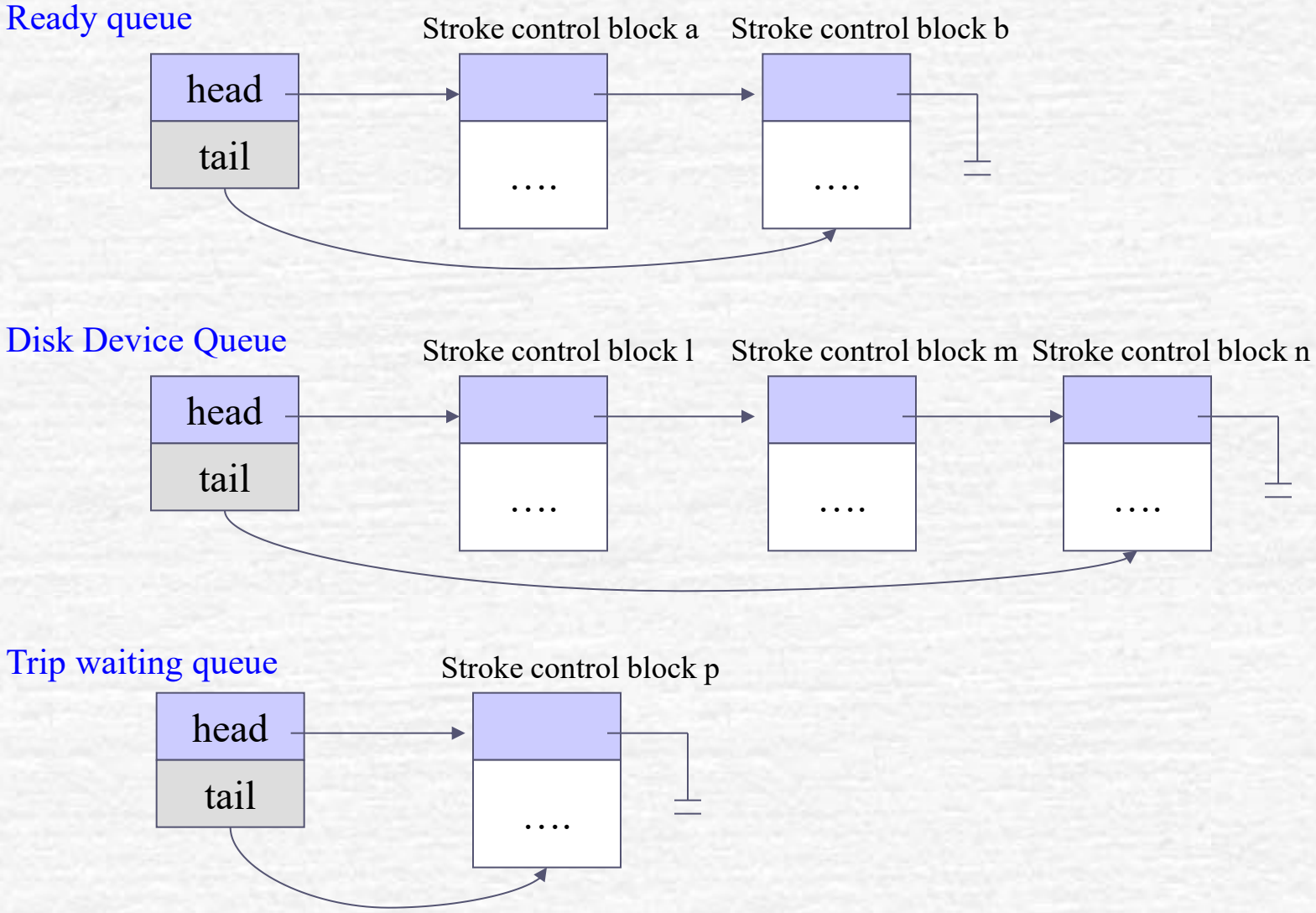
- Processes are the basic unit of operating system management and allocation of resources
- The operating system stores the control information of the process in the Process Control Section (PCB).



Queue for the Process

- The operating system uses queues to help with scheduling
- Job Queue: Used to record all processes in the system
- Ready Queue: Used to store the process in the ready state
- Device Queues: To wait for the queue generated by the completion of I/O, each device has its own corresponding device queue
- Wait Queue: Stores processes that are running system calls or waiting for events to occur

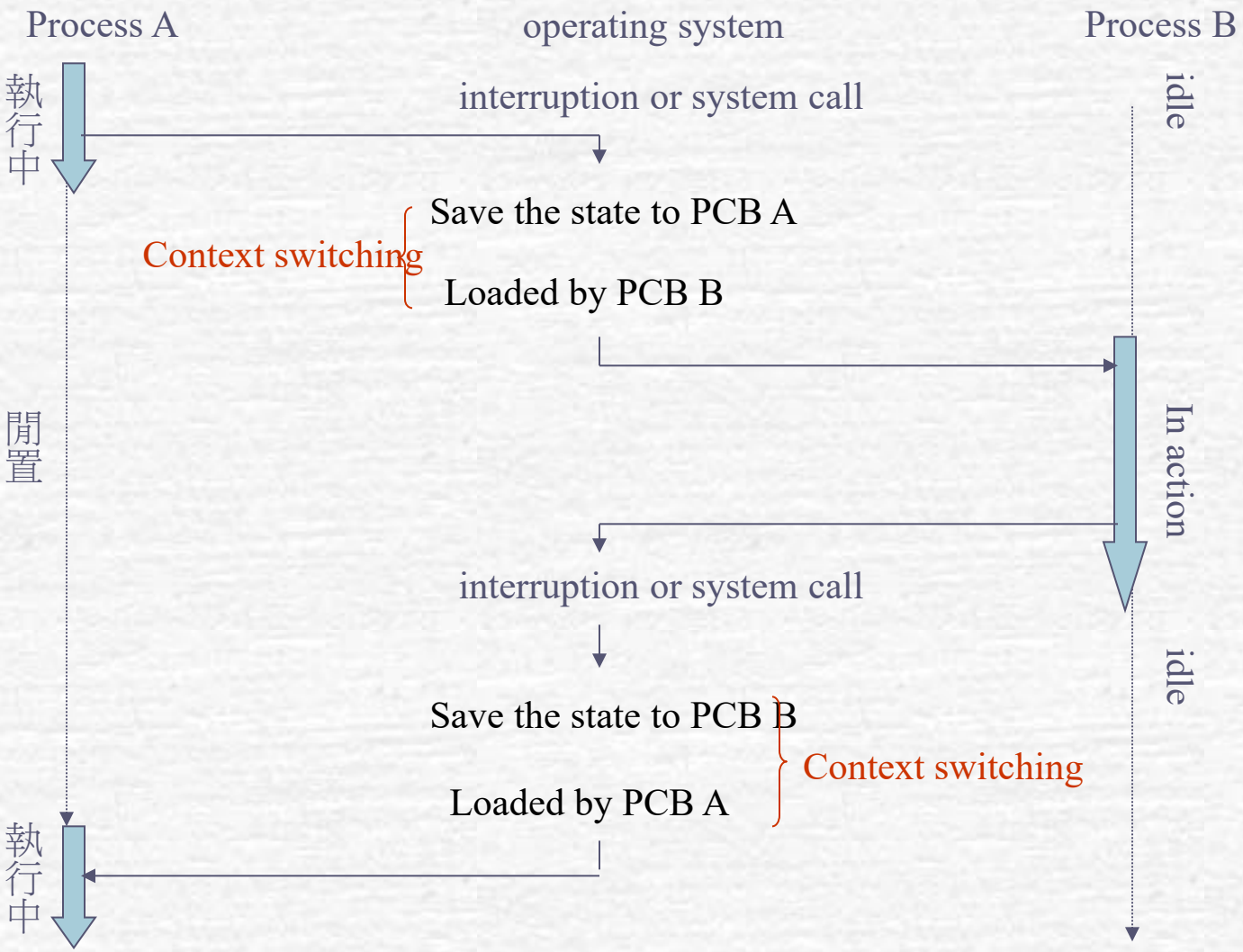
Figure 2-8 Ready queue and device queue



Context switching

- Context switching: When the CPU is switched from one process to another, the operating system must store the information about the process in the process control section of that process, and load the program control section of the other process into the system in order to restore the execution environment to the state it was in when the latter was interrupted

Context switching



Context switching and interrupting

- Context switching of a process requires the operating system to regain control first
- The operating system usually regains control through an outage
  - Events that are not related to the currently running process are often referred to as interruptions, such as clock interrupts and input/output interrupts
  - Events related to the currently running process, including exception interruptions and software interruptions
- In terms of system utilization, the time spent on body switching can be said to be a pure waste because it is not doing productive work
  - The speed of body switching will vary depending on the computer hardware architecture
  - The more complex the operating system, the more tasks need to be performed when switching between contexts

Threads - Lightweight processes

- Thread: The basic unit of CPU configuration
  - It has its own program counters, registers, and stack space
  - It shares the same memory address space, program segments, data segments, and some system resources as other threads in the same process
- A traditional process is equivalent to a single-threaded process with only one thread
  - The disadvantage of a single-threaded process is that when it receives multiple requests at the same time, it either executes sequentially or has to generate multiple sub-processes to provide better interaction
  - This approach requires frequent creation of a large number of processes and body switching, which not only consumes system processing time, but also takes up a piece of memory space for each process
- Many server programs today use multithreading to improve performance

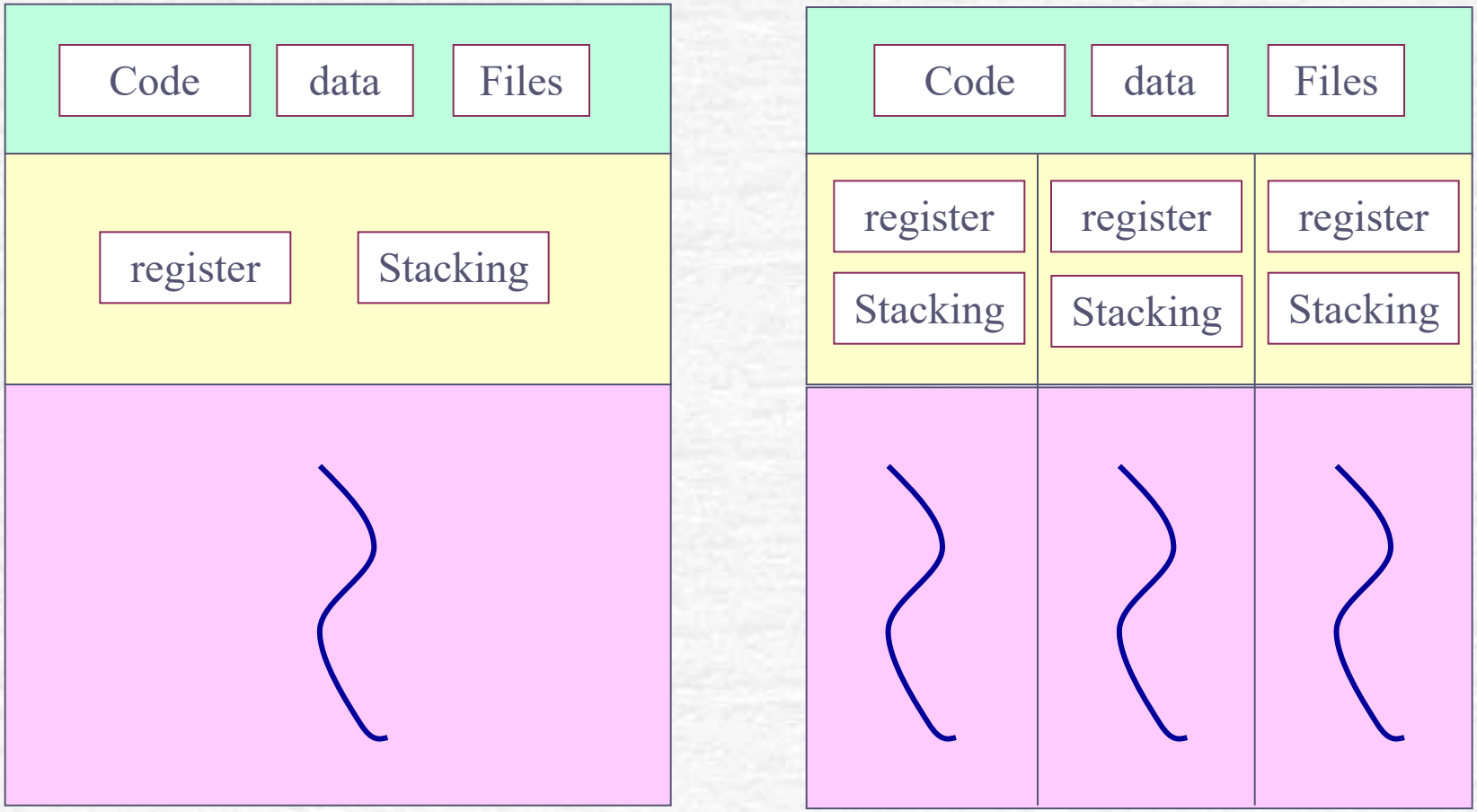


Figure 2-10 Single-threaded and multi-threaded processes

The main difference between a process and a thread

- Address space: The address space between processes is independent of each other, while threads belonging to the same process share the same address space.
- Communication mode: communication between processes must be carried out using the mechanism provided by the operating system; Because threads share the same address space, they can communicate by directly reading and writing their global data.
- Body switching: The time for body switching between threads in the same process is much smaller than that for body switching between processes.

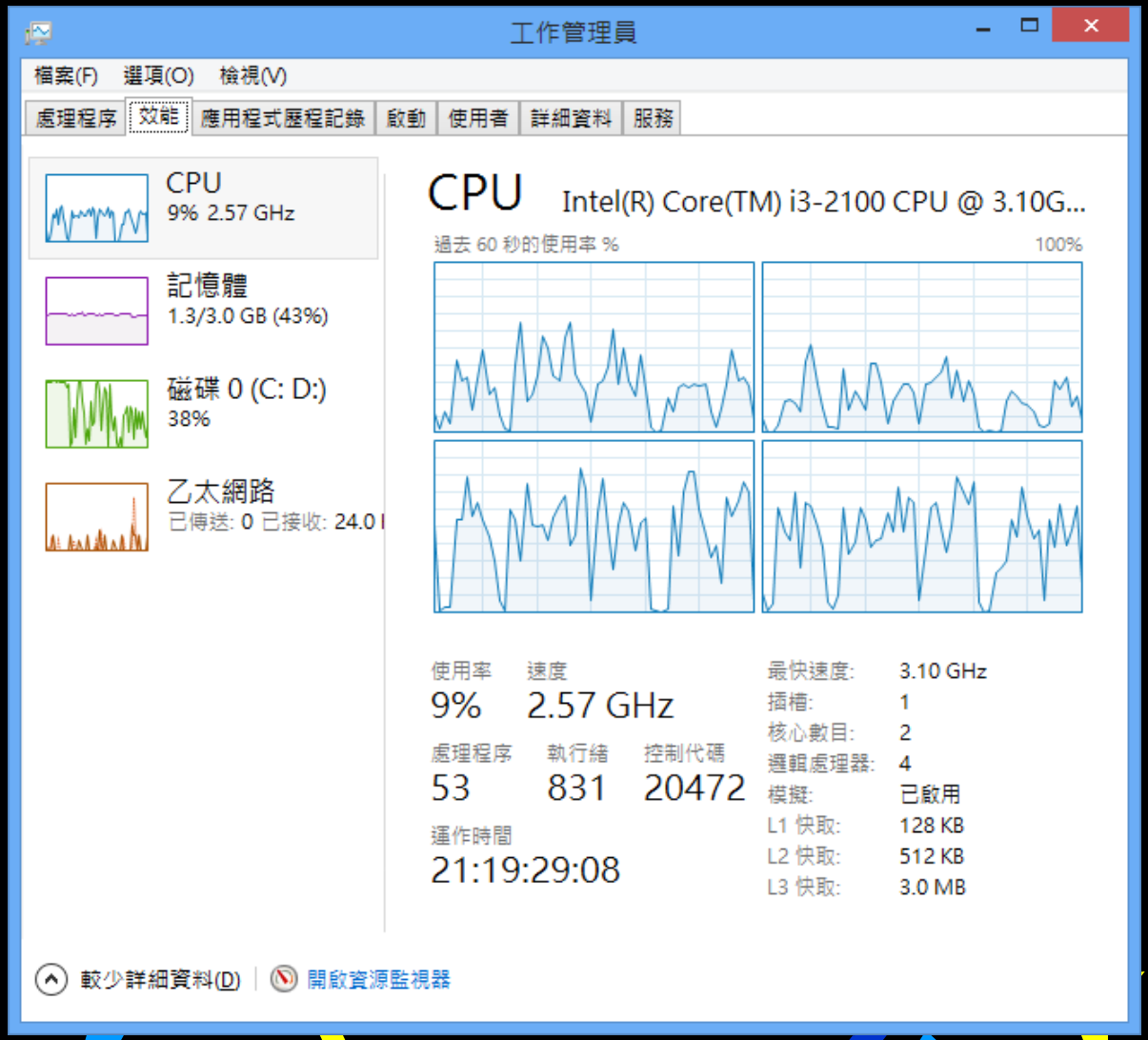
Advantages of using multithreading

- Quick response
- Resource opening
- Efficiency
- Parallel processing

Observe the number of processes and threads in the system

- Press Ctrl-Alt-Del to invoke the Windows Task Manager
- Switch to the Performance tab

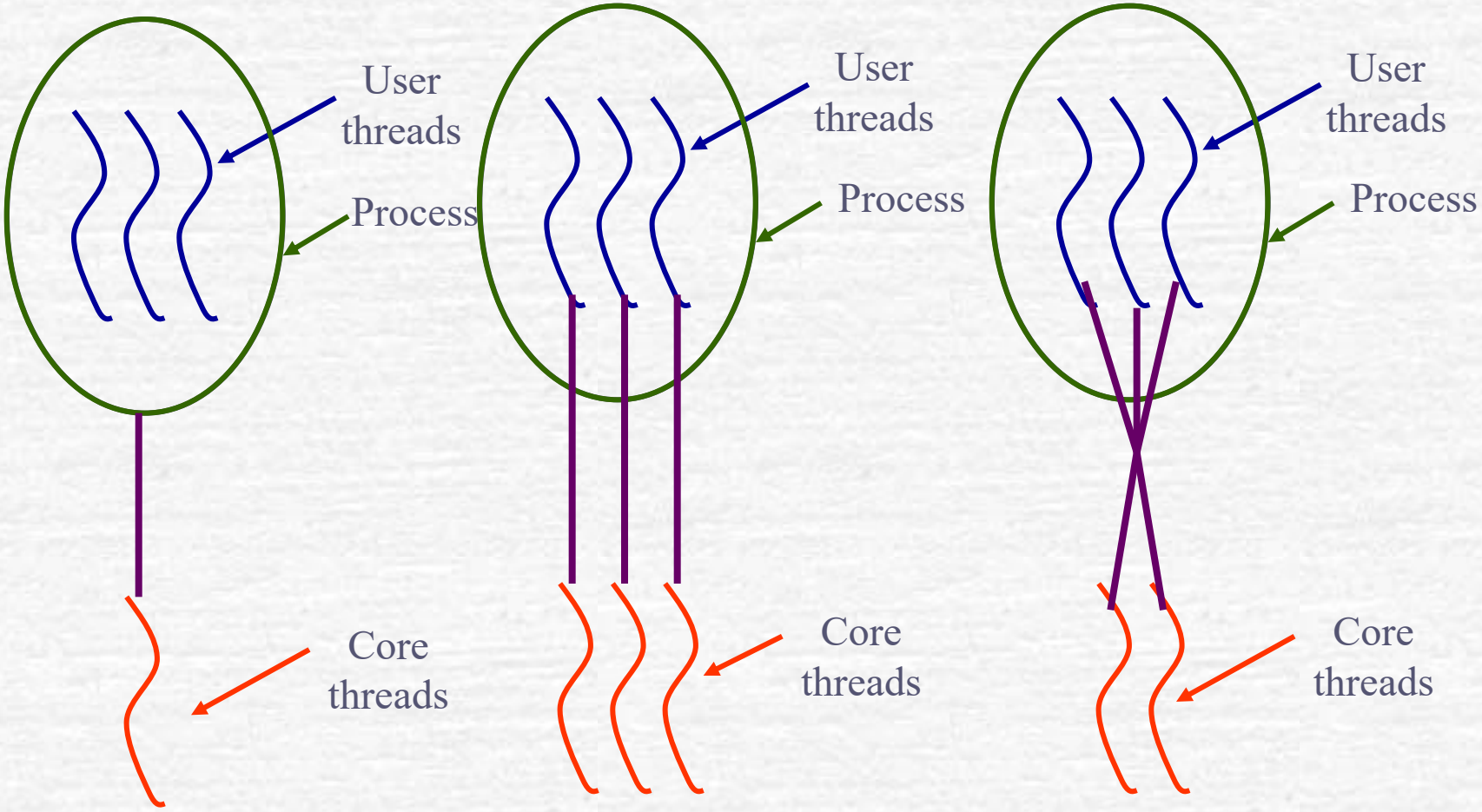
Figure 2-11 Process and thread information displayed by the Windows Task Manager



Implementation of threading functionality

- System threads: The operating system kernel supports and manages the state and information of each thread
- User thread: The thread that manages it is tracked by the user process

There are three mappings between user threads and core threads



a. Many-to-one mode

b. One-on-one mode

c. Many-to-many mode

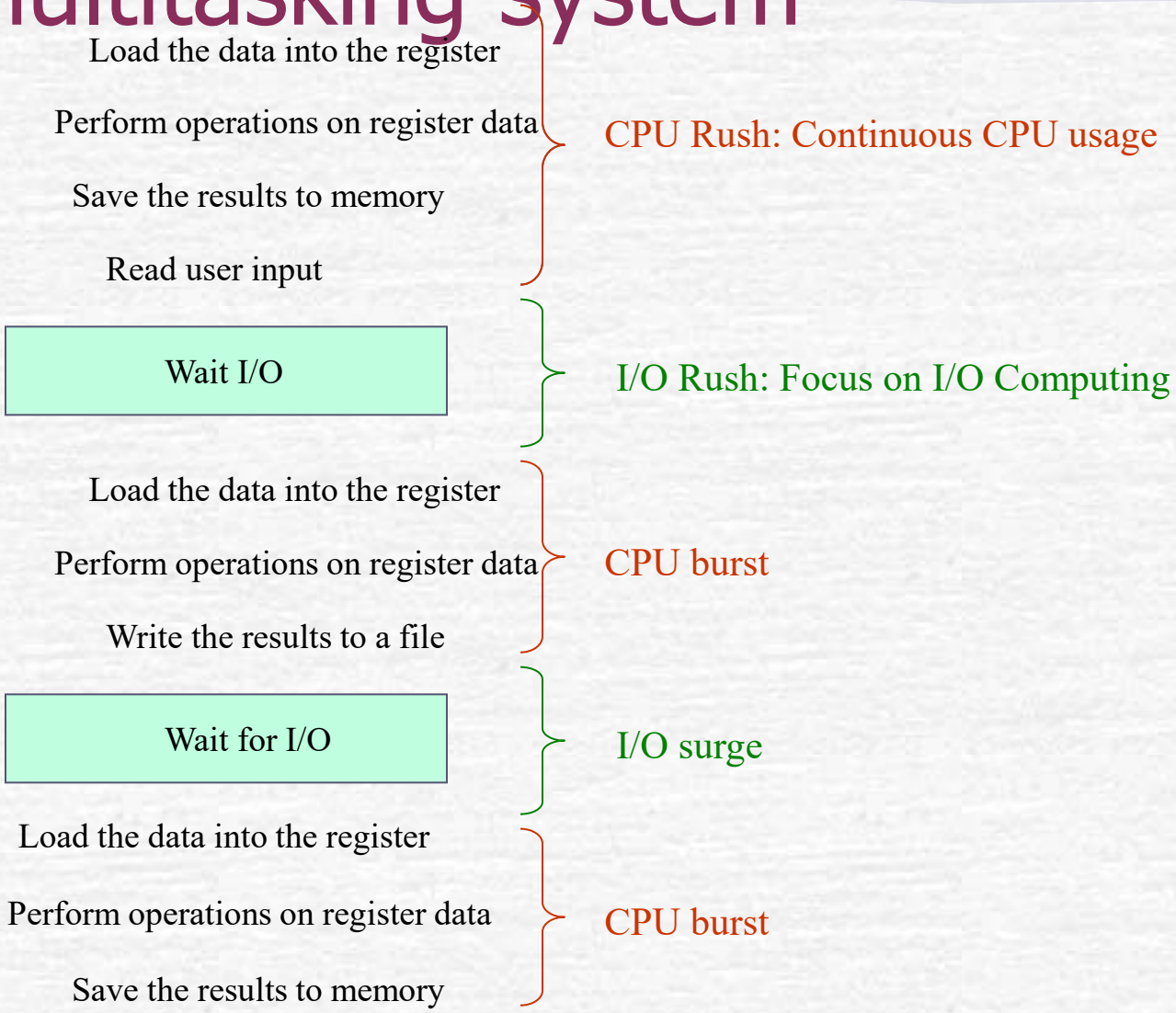
thread pool

- Some threads are generated when the process starts to run, and then put into the thread pool to wait for work
- When the server receives a request, it only needs to wake up a thread in the pool to process it
- After the thread completes its work, it returns to the pool and waits
- Pros：
  - The efficiency is better
  - It is easy to control the number of threads

2-5 Schedule of Process

- Process scheduling is the foundation of a multitasking system
- Concept of scheduling: When a process is waiting for an event or I/O operation, because the CPU cannot be used, the CPU can be given up to other processes that need to be executed
- The execution of the process is usually constantly switched between the two conditions：
  - CPU Rush: Continuous CPU usage
  - I/O Rush: Focus on I/O calculations

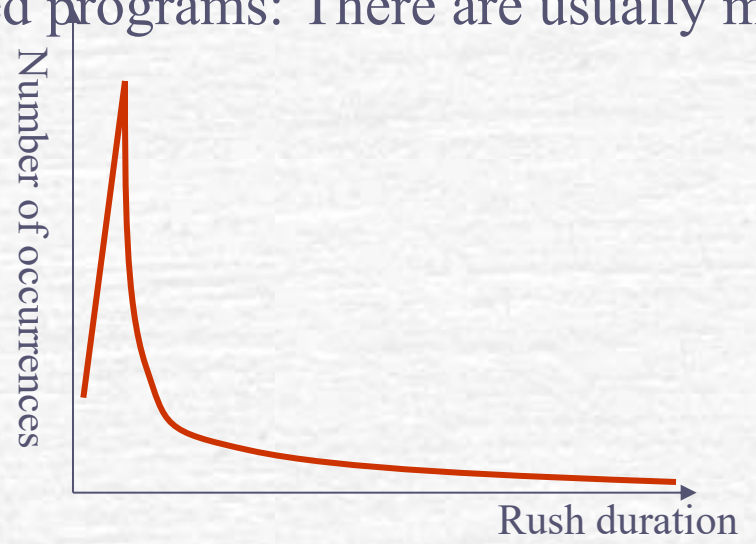
Trip scheduling is the foundation of a multitasking system



When an I/O burst occurs, the stroke enters a suspended state

CPU burst distribution

- CPU burst times for different computer architectures and different processes often vary considerably
- But in principle, most of them have a large number of extremely short CPU bursts, while the longer CPU bursts have much fewer numbers
- This distribution of CPU burst is very important data for how to select or design appropriate CPU scheduling algorithms
  - CPU-Focused Programs: Requires a lot of CPU computing
  - I/O-focused programs: There are usually many short CPU bursts



Measure of scheduling performance

- The purpose of scheduling is to pick out the one that delivers the best performance out of all the ready schedules
- Some common scheduling criteria
  - CPU Usage: The proportion of time that the CPU is actually executing the process
  - Capacity: The number of trips that the system can complete per unit of time
  - Reply time: The average time it takes for a single trip to complete
  - Waiting time: The average time the trip spends waiting
  - Response time: In an interactive system, the average time it takes for the system to start responding after the user inputs
  - Predictability: Used to assess the consistency of the system's response
  - Fairness: The degree to which all trips have the same chance of execution

Front-running and non-front-running scheduling

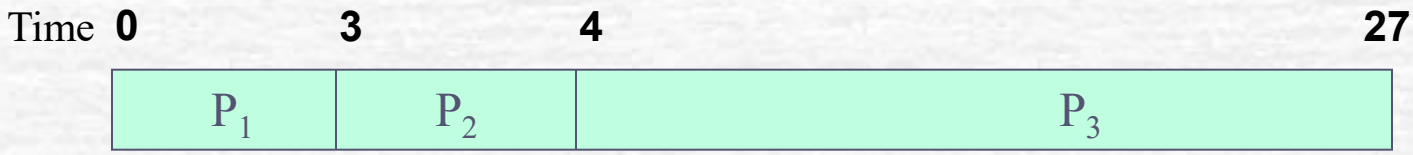
- Non-preemptive scheduling: Only when a running process enters a suspended state or terminates can other processes gain control of the CPU
  - It allows the running process to complete the entire CPU burst
  - In modern interactive systems, this can make other users or applications wait too long
- Preemptive scheduling: The scheduler can kick a trip out of the CPU before it enters a wait or ends, so that the CPU can be allocated to another process
  - The current system adopts the practice
  - Requires more complex CPU design

Schedule on a first-come, first-served basis（FCFS Scheduling）

- No preemptive scheduling method
- Pros: Simple
- Cons: Wait times are highly variable, and the average wait time is not short and unpredictable
- FCFS scheduling is more beneficial for CPU-based programs
- Escort phenomenon can occur: a situation where all processes are waiting for a large Process to leave the CPU

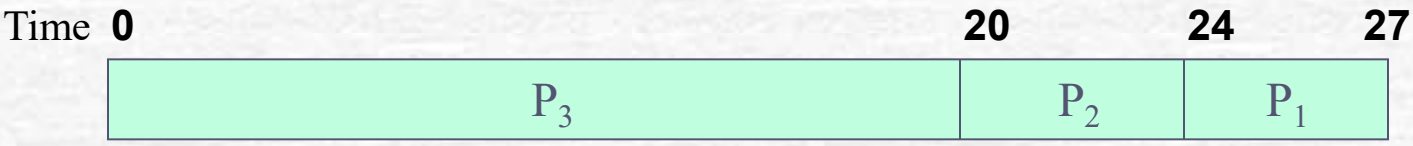
Figure 2-16 Example of first-come, first-served scheduling

Process	CPU burst time（milliseconds）
P_1	3
P_2	4
P_3	20



(a) Make requests in the order of P1, P2, P3

Average wait time= (0+3+7)/3 = 3.3 milliseconds



(b) Request in the order of P3, P2, and P1

Average wait time= (0+20+24)/3 = 14.7milliseconds

Prioritize the shortest task schedule （SJF Scheduling）

- It is not possible to preemptively schedule
- I hope to choose the shortest stroke for the next CPU burst
- It first appeared in batch processing systems, relying on the estimated time provided by batch jobs for scheduling
- The current practice is to use the arithmetic average or exponential average of the past burst time of the process as an estimate of the next CPU burst
- $e_{n+1}=\alpha t_n + (1-\alpha)e_n$

  t_n ：The time of the nth rush，e_n ：The expected time of the nth surge

  α ：The weight of the most recent surge time

Figure 2-17 Example of the shortest task priority schedule

Process	CPU burst time（milliseconds）
P_1	20
P_2	4
P_3	3

Average wait time=3.3 Milliseconds (FCFS Best Case)

Characteristics of SJF scheduling

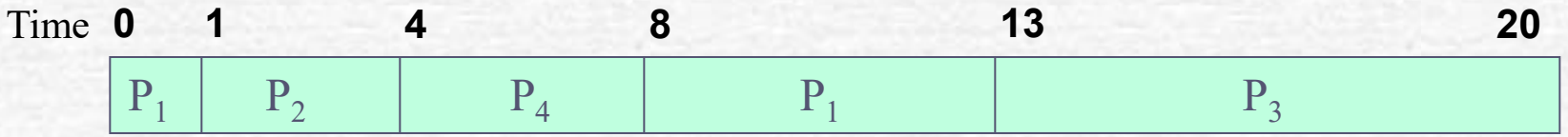
- The one with the lowest average wait time of all non-front-running scheduling methods
- Delay the execution of processes with long CPU bursts to reduce average latency
- If there are constantly CPU bursts of short processes entering the ready queue, it may cause the CPU to burst with longer CPU bursts and starvation phenomenon and never be able to gain control of the CPU

Minimum remaining time schedule (SRT Scheduling）

- Early access to SJF scheduling
- When a new process enters the ready queue, if the CPU burst time is shorter than the CPU burst time left for the currently running process, the currently running process is kicked out and the new process is preempted by the new process
- Long CPU bursts can also cause starvation

Figure 2-18 Minimum Remaining Time Schedule Example

Process	CPU burst time (ms)	Arrival time
P_1	6	0
P_2	3	1
P_3	7	2
P_4	4	3



Average wait time= ((8-1) + (1-1) + (13-2) + (4-3)) / 4 = 4.75 milliseconds

(a) SRT scheduling results

average wait time= ((0-0) + (6-1) + (13-2) + (9-3)) / 4 = 5.5 milliseconds

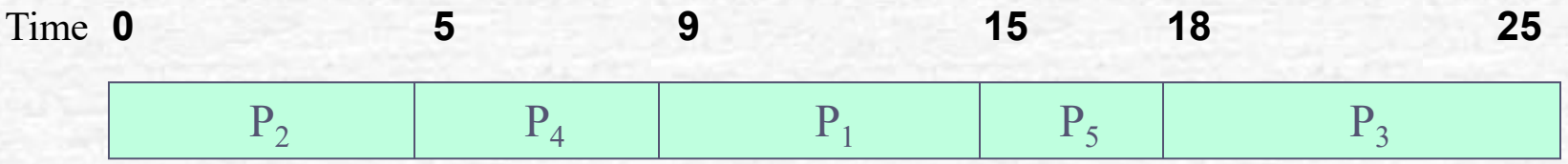
(b) SJF scheduling results

Priority scheduling
（Priority Scheduling）

- Non-preemptive / Preemptive / Preemptive
- The scheduler schedules based on the level of priority
- Criteria for specifying priority：
  - Characteristics of the itinerary
  - User level
  - Some parameters specified
- Some priority designation mechanisms allow priority to change dynamically, while others are fixed priority designs
- The SJF scheduling method can also be considered a special case of priority scheduling
- It may cause starvation
- The aging mechanism can be used to solve the hunger phenomenon

Figure 2-19 Priority scheduling example

Process	CPU burst time (ms)	Priority
P_1	6	2
P_2	5	0
P_3	7	3
P_4	4	1
P_5	3	2



Average wait time= (9+0+18+5+15)/5 = 9.4 milliseconds

Cyclic time-sharing scheduling（Round-Robin Scheduling）

- A scheduling method specially designed for time-sharing systems
- Similar to FCFS, small unit early access version
- In principle, the process that has been waiting in the queue for the longest time will be allowed to enter the CPU execution, but after a fixed period of time, the scheduler will interrupt the running process
- The process may be interrupted during the CPU burst, and this unit of time is called time slicing

Advantages of RR scheduling

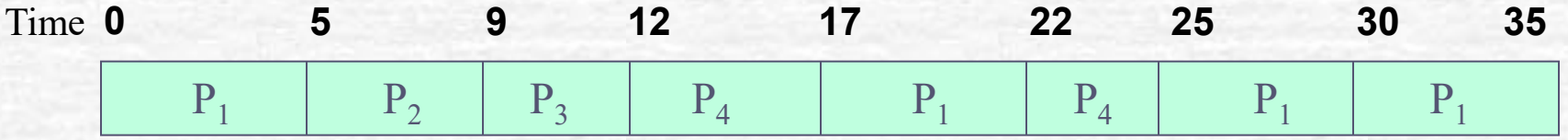
- When there is a CPU-based process in the system, there will be no escort phenomenon
- The average waiting time is longer, but it provides a better response time
- The length of the time slice：
  - If the slicing time is too short, the frequency of body switching will be too high, which will affect the system performance
  - If the slicing time is too long, it will approximate the effect of FCFS scheduling, causing a similar escort phenomenon
  - A general rule of thumb: 80% of CPU burst time should be less than the length of a time slice

Figure 2-20 Example of recurring time-sharing schedule

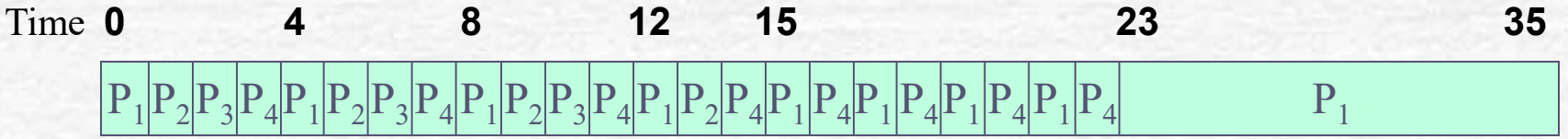
Process	CPU burst time (ms)
P_1	20
P_2	4
P_3	3
P_4	8

Average wait time= ((25-10) + 5 + 9 + (22-5)) / 4 = 11.5 milliseconds

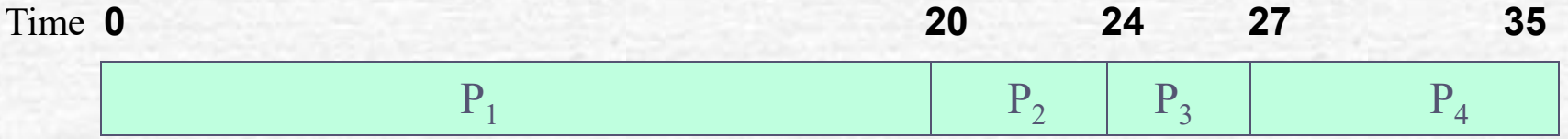
a. Time slices= 5（milliseconds）



b. Time slices= 1（ms)



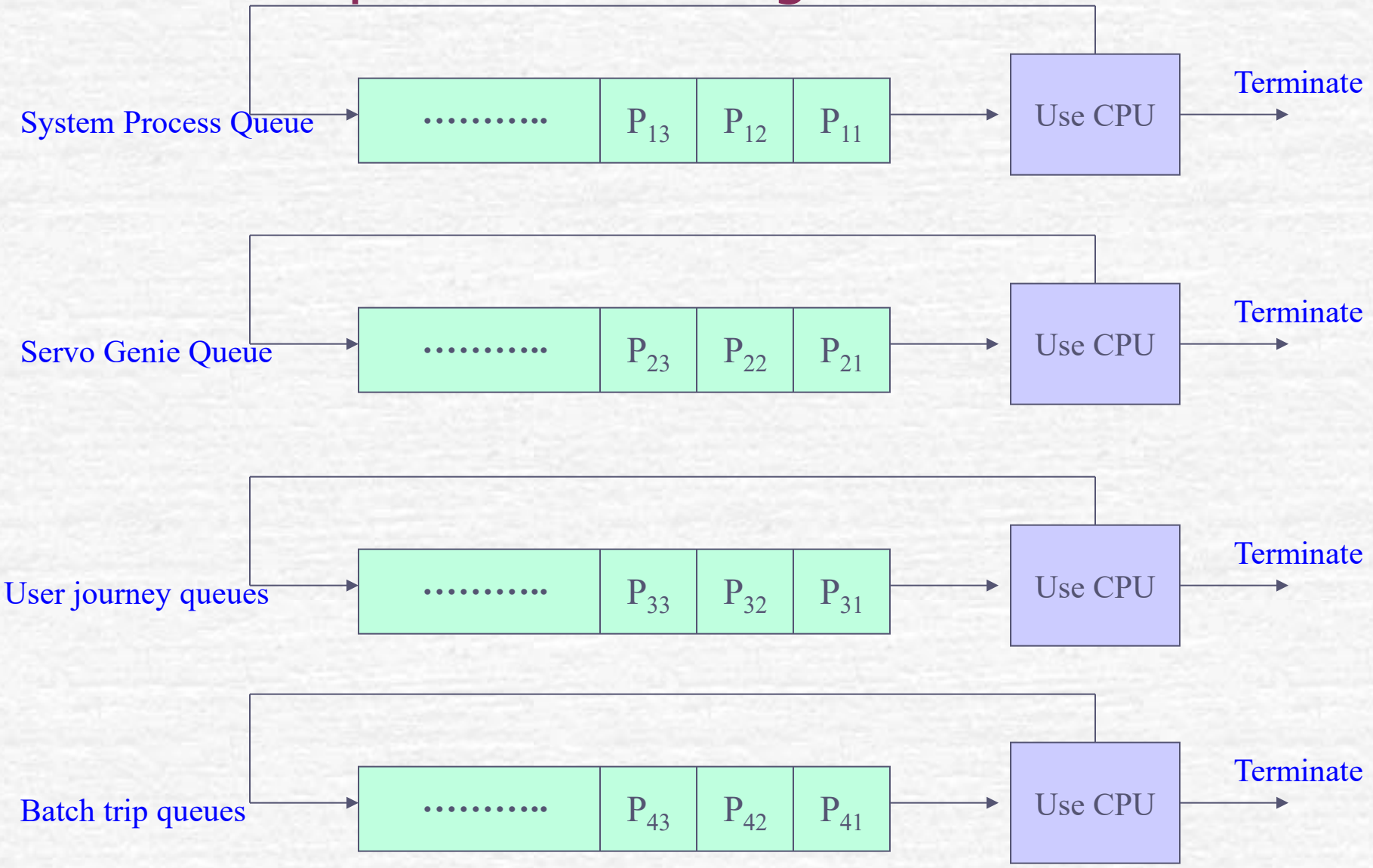
c. Time Slice = Infinity→FCFS



Multi-level queue scheduling
（Multilevel Queue Scheduling）

- Categorize the journeys, place different types of trips in different queues, and use the scheduling method that best suits that type of trip within each queue
- For example, divide the itinerary into foreground and background itineraries
- In addition to the scheduling method within the queue, there is a need for a holistic scheduling approach between queues
- Commonly used inter-queue scheduling methods are:：
  - Front-running fixed-priority scheduling method
  - A scheduling method similar to RR

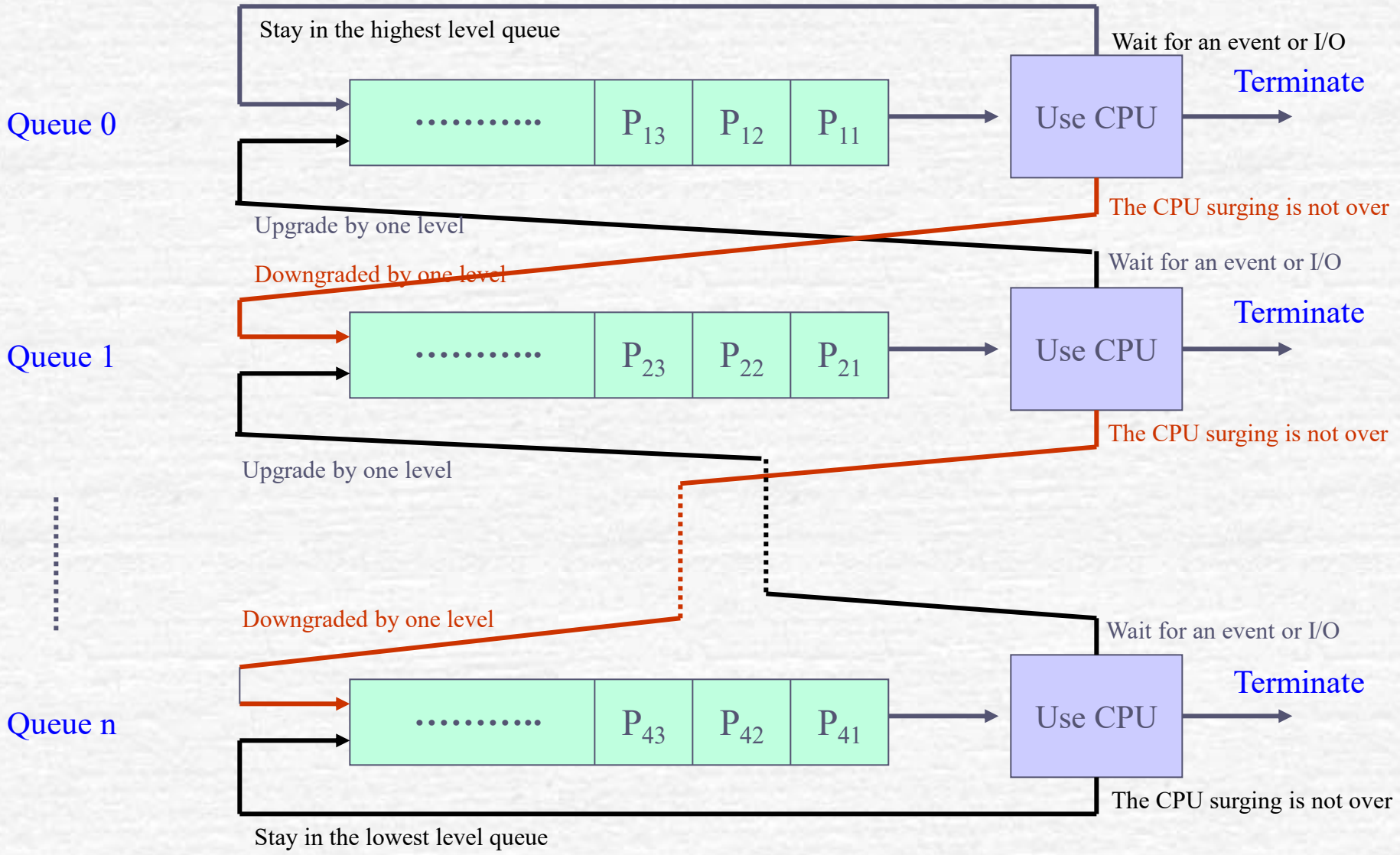
Figure 2-21 Schematic diagram of multi-tier queue scheduling



Multi-tiered feedback queue scheduling

- Priority scheduling that can be preempted
- Depending on the time of the CPU burst, the process is dynamically moved between queues
- I/O-focused trips typically stay in higher-level queues
- Once CPU-focused processes gain CPU control, they can be allocated to longer time slices
- Starvation can occur and can be avoided by using aging mechanisms

Figure 2-22 Schematic diagram of multi-tier feedback queue scheduling



Classroom Exercise: Scheduling Method (1)

- Suppose there are four trips, P1, P2, P3, and P4, all arrive at time 0 in the order of P1, P2, P3, P4
- Please use the preemptive scheduling method for FCFS, SJF, and non-preemptive priority scheduling：
  1. Please draw a diagram of the sequence of the itinerary similar to the text
  2. Calculate the average wait time for all trips
  3. What is the response time for each trip?

Process	CPU burst time（ms）	Priority （0 has the highest priority）
P1	7	1
P2	5	0
P3	3	3
P4	4	1

Exercise 1 solution-FCFS

FCFS

P1	P2	P3	P4

Average wait time：(0 + 7 + 12 + 15) / 4 = 8.5

Response time：

P1	7
P2	12
P3	15
P4	19

Exercise 1 solution：SFJ

SJF

P3	P4	P2	P1

Average wait time：(0 + 3 + 7 + 12) / 4 = 5.5

Response time：

P1	19
P2	12
P3	3
P4	7

Exercise 1 Answer: Priority

Average wait time：(0 + 5 + 12 + 16) / 4 = 8.25

Response time：

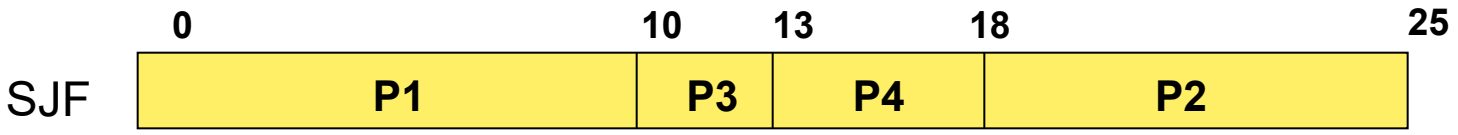
P1	12
P2	5
P3	19
P4	16

Classroom Exercise: Scheduling Method (2)

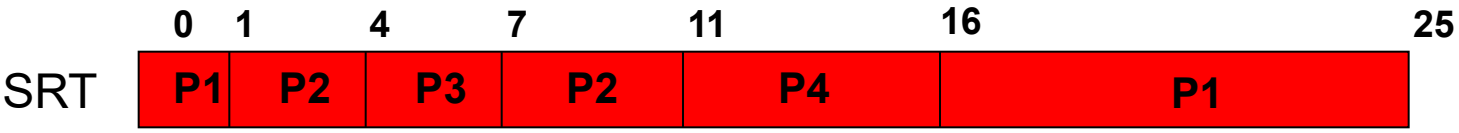
- Suppose there are four trips, P1, P2, P3, and P4, arriving at different times
- Please use the SJF and SRT scheduling methods:
- Please draw a diagram of the sequence of the itinerary similar to the text
- Calculate the average wait time for all trips

Trip	CPU Burst Time (ms)	Arrival time (0 has the highest priority)
P1	10	0
P2	7	1
P3	3	4
P4	5	4

Exercise 2 Answer



Average wait time：(0 + 7 + 12 + 15) / 4 = 8.5



Average wait time：(15 + 3 + 0 + 7) / 4 = 6.25

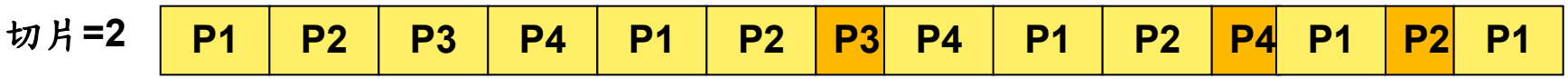
Classroom Exercise - Scheduling Method (3)

- Suppose there are four trips P1, P2, P3, and P4 that all arrive at time 0, in the order of P1, P2, P3, P4
- Please use the RR scheduling method for time slices of 2, 5, and 10：
  1. Please draw a diagram of the sequence of the itinerary similar to the text
  2. Calculate the average wait time for all process

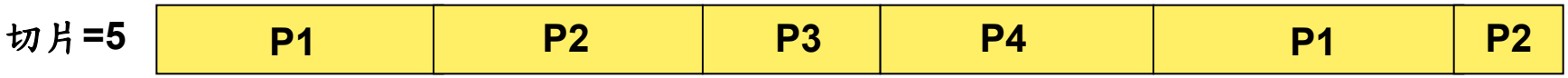
Trip	CPU Burst Time (ms)
P1	10
P2	7
P3	3
P4	5

範例補充包-Extra~

Exercise 3 Answers



Average wait time：(15 + 10 + 15 + 16) / 4 = 14



Average wait time：(13 + 18 + 10 + 13) / 4 = 13.5

Average wait time：(0 + 10 + 17 + 20) / 4 = 11.75

Scheduling algorithm for Linux

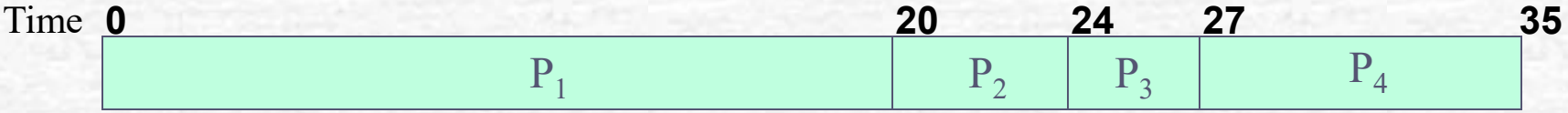
- Linux adopted the CFS scheduling algorithm after version 2.6.23
  - Completely fair scheduler(Completely Fair Scheduler)
  - Basic concept: To simulate a "ideally" very accurate multitasking CPU on real hardware CFS algorithm implementation
  - Nanoseconds are used as the unit of calculation of time
  - Use the RBTREE to manage the schedule of your trips. The less CPU time is allocated, the more it will move to the left of the tree. When a process enters the CPU to execute a time slice, the scheduler compares the total time used by the process with the current used time of the leftmost node of the red-black tree to determine the next running process
  - Under the "ideal multiplexer" principle, CSF still calculates the "virtual execution time" allocated to each process by considering priority and nice values

Performance evaluation of scheduling algorithms

- Qualitative mode
- Queuing theory
- System simulation

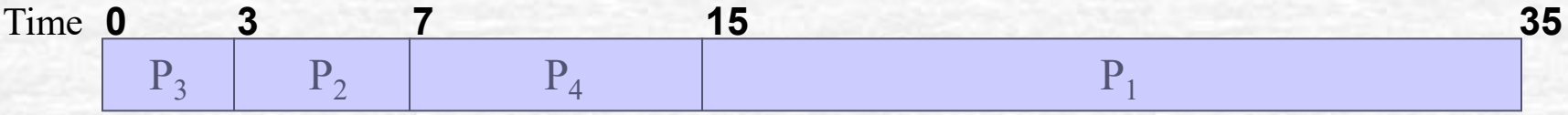
Figure 2-23 Example of algorithm performance evaluation for qualitative mode

Trip	CPU burst time (ms)
P_1	20
P_2	4
P_3	3
P_4	8



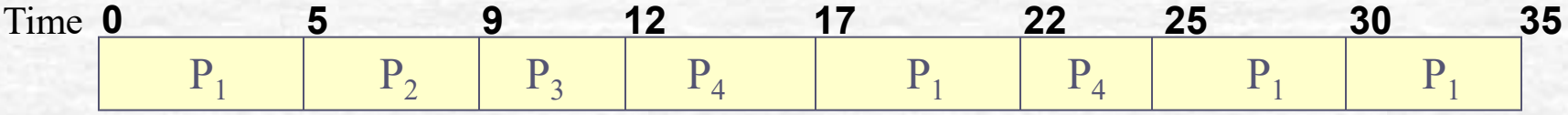
(a) FCFS scheduling results

Average wait time= (0 + 20 + 24 + 27) / 4 = 17.75 ms



(b) SJF scheduling results

Average wait time = (0 + 3 + 7 + 15) / 4 = 6.25 ms



(C) RR Scheduled Result (Time Slice = 5 ms)

Average wait time = ((25-10) + 5 + 9 + (22-5)) / 4 ms

Thread scheduling

- When a process contains multiple threads, two schedule units appear: process and thread
- Different levels of thread implementation also affect thread scheduling considerations
  - User threads：
    - Schedule on a schedule-by-trip basis using the original scheduling algorithm
    - At runtime, its thread scheduler picks out a thread to run based on its own scheduling policy
    - There is no way to force an interrupt for threads that have been running for too long
  - Core threads：
    - The operating system can choose one of all threads to run
    - In a multiprocessor system, it is possible for multiple core threads belonging to the same process to run on different CPUs simultaneously. In this case, data inconsistencies can occur

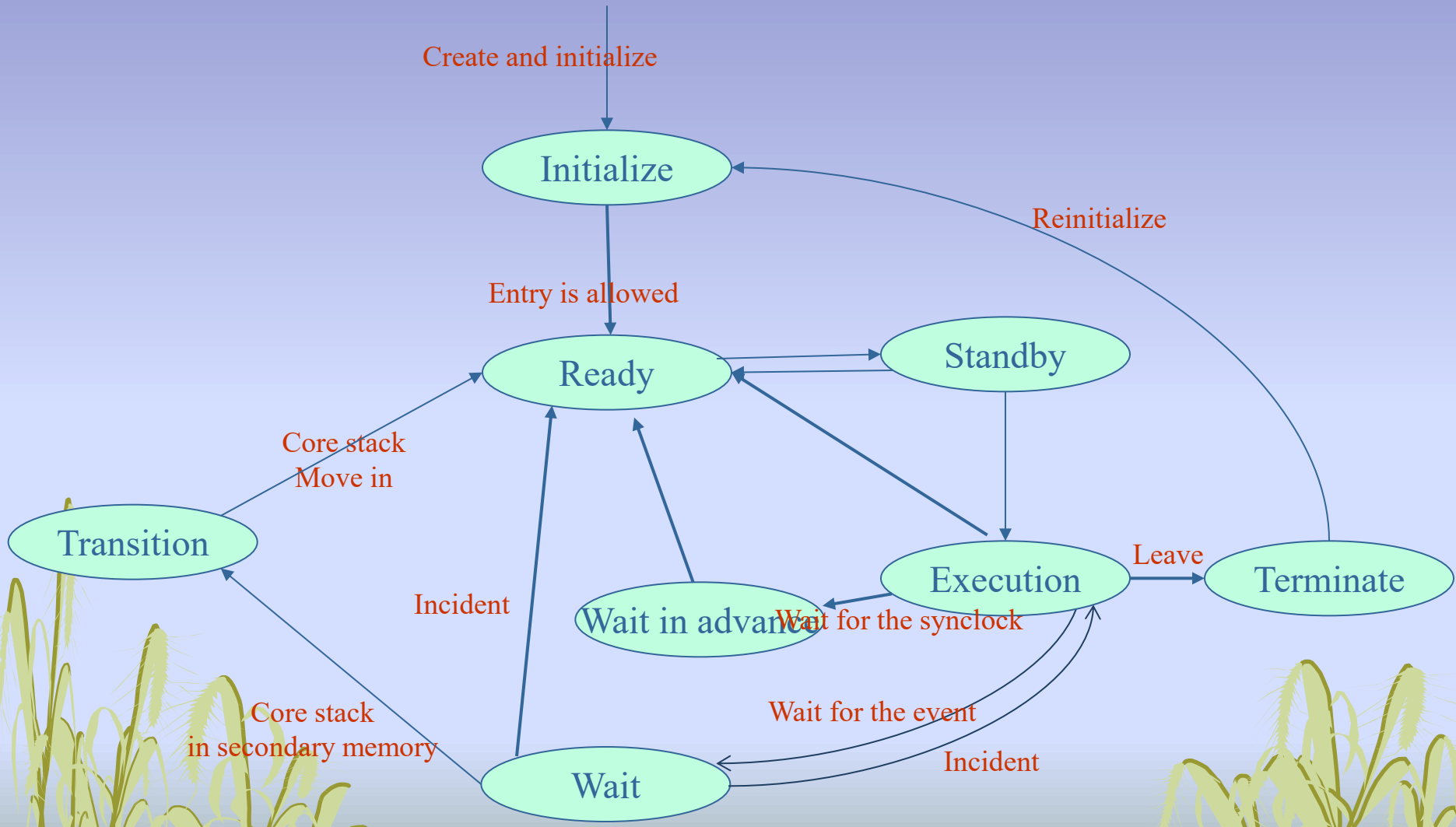
Windows Thread Status (Optional)

- Initialize：Threads are still being created.
- Ready：Indicates that the thread has acquired the required resources other than the processor and is waiting for a schedule。
- Standby：The next thread to run
- Execution：Entered the execution state
- Wait：A thread is waiting for an event
- Transition：While a thread is waiting, it transitions to a transitional state if its core stack is in secondary memory
- Terminate：When the thread is executed, the system can reinitialize the thread object

Windows Thread Status (Optional)

- Wait in advance：This is to support the state that the synchronization mechanism is designed to support in a multi-core or SMP architecture. If a thread running in a CPU core finds that the synchronization mechanism it needs has been occupied by other threads, it will enter the early waiting state and release the CPU core for other threads to use. This avoids the impact on performance caused by threads busy waiting in the processor. According to the results of the experiment, this design of Windows 7 still performs well even on a multi-core platform with 256 cores.

Figure 2-24 Windows thread status



Windows thread scheduling

- Priority-based, preemptible scheduling and implemented with multi-tiered queues
- When a running thread releases CPU control：
  - Being preempted by other higher priority threads
  - Time slice expires
  - The mission execution ends
  - Wait for an event to happen

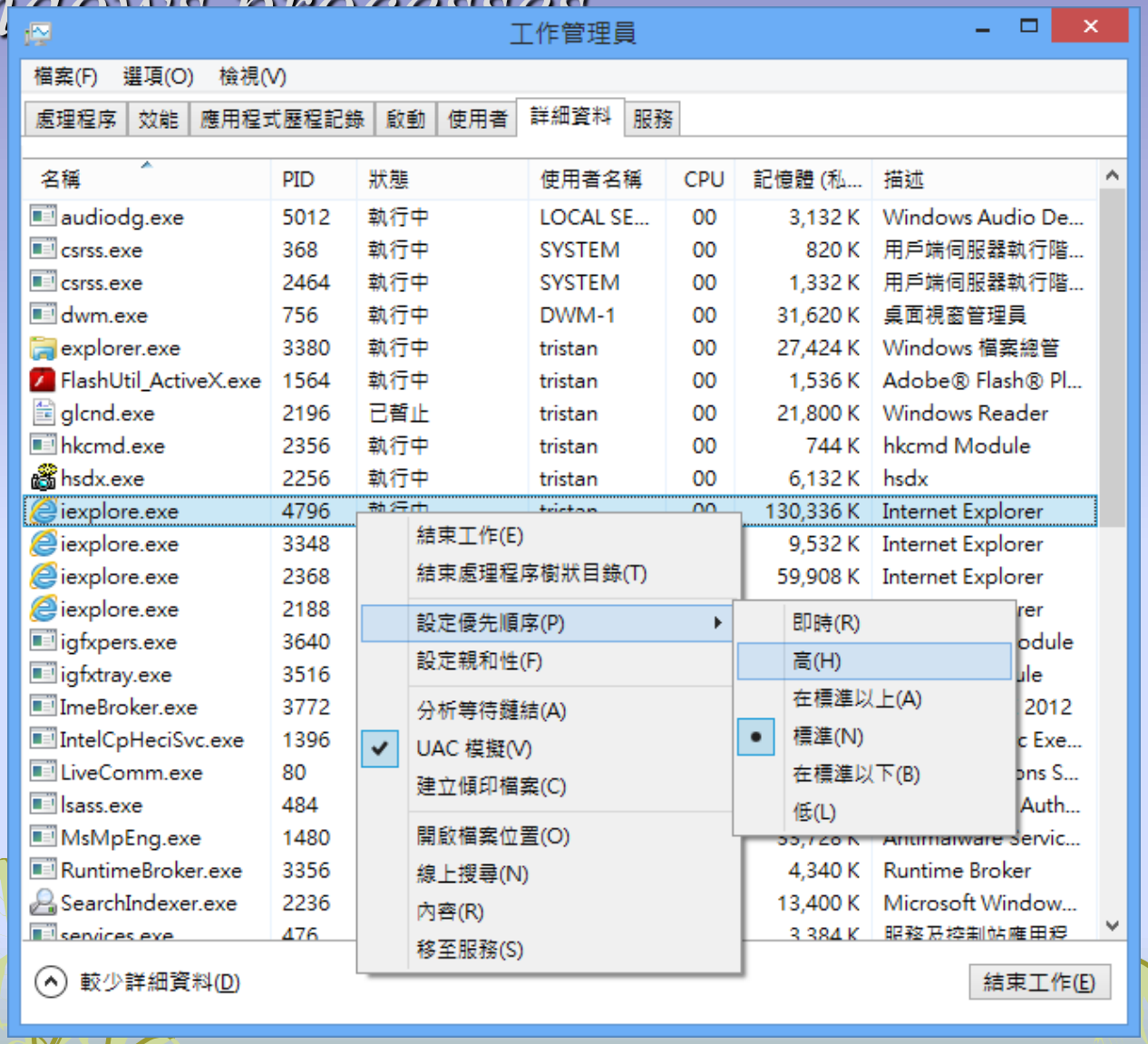
The priority class of Windows threads

- There are two main categories of priorities in Windows：
  - Variable Categories：0 to 15
  - Real-time categories：16 to 31
- Users can increase or decrease the priority of threads within a certain range
- If there are no currently ready threads in the system, the dispatcher goes to run the idle thread

The runtime quota for the thread

- Each thread in Windows will have a time quota for this execution, which represents an integer multiple of the basic quota unit
- When a running thread encounters a timer interrupt, the time quota is reduced (minus 3)
- Windows increases the time quota for foreground threads to provide better response times
- Users can use the task manager to modify the priority of their schedules

Figure 2-25 Setting the priority of Windows processes



The scheduling algorithm for Windows threads

- Windows uses a dynamic priority scheduling methodology
- When an event that a thread is waiting for occurs, priority is usually temporarily increased, but the thread's time quota is reduced by 1 to avoid being too favorable to some threads
- Essentially, the degree of priority elevation of a thread is proportional to the I/O response time it requests
- When a thread in a running state runs out of time quota, Windows determines whether it needs to be lowered
- If a thread with a higher priority is preempting the CPU during execution, the original thread will return to the beginning of the queue at the same level